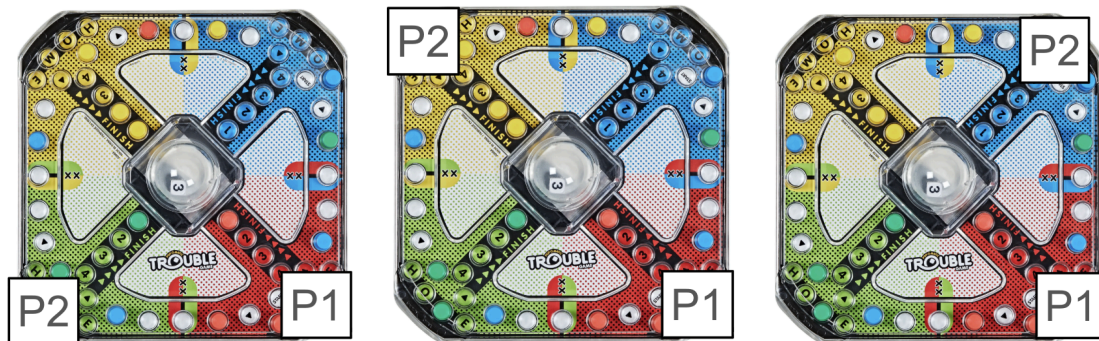


Trouble Strategy Simulation and Analysis - Project Rundown

Original Motivation

This project originated from the board game 'Trouble'. While deciding player order, the question of 'does going first provide an advantage' came up. The original project was just an online simulation where I had planned to simulate thousands of games of Trouble and explore if starting position gave an advantage. I also decided to dive into positional advantage in 2-3 player matches. For example:



Which of these starting situations would grant an advantage to which player?

Strategy Development

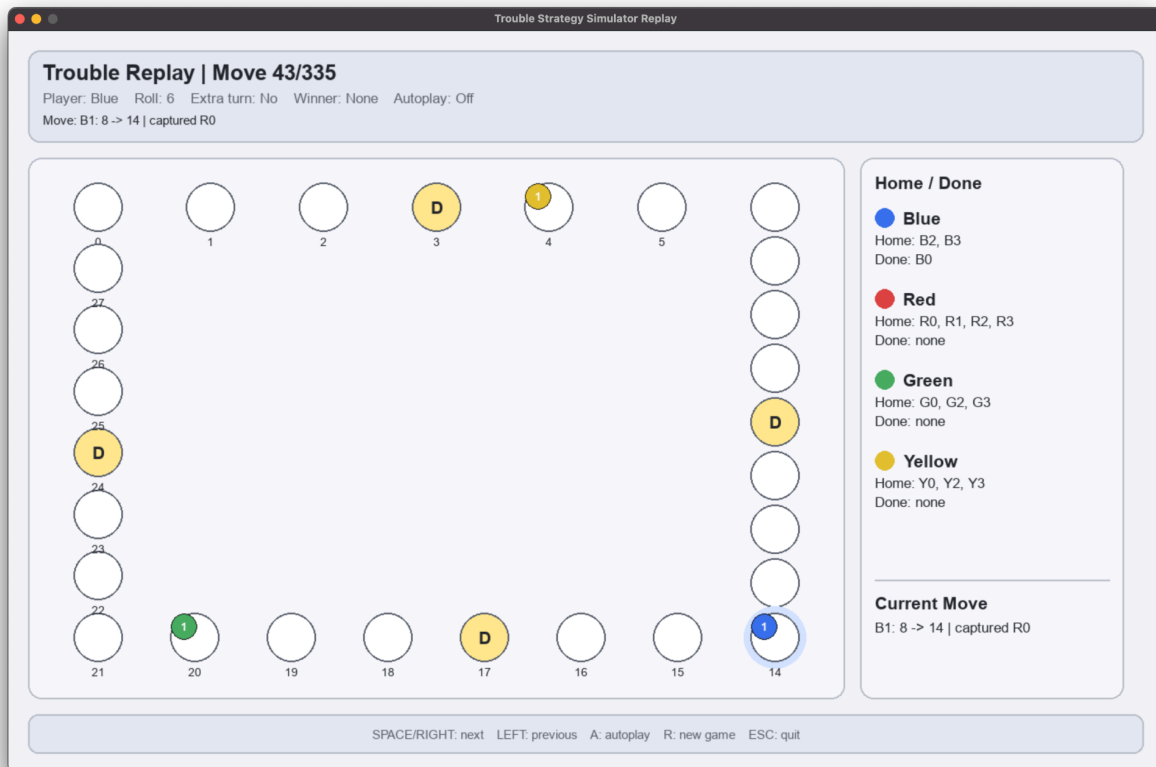
I soon realized that to gain valid insights, I'd have to simulate games as close to real-life as possible. An advantage seen in games where all moves are random may not be reflected in real-life games. So, I developed multiple strategies, and first simulated games in which every player played the same strategy. I ended up getting mixed results between different strategies, and the small advantages seen in win percentage were very incremental.

Strategies Applied (trouble_sim/strategies.py):

- Random - a random (valid) piece is chosen to move forward
- Furthest - the piece that is furthest along is moved
- Capture - same as furthest, but prefer to capture (if possible)
- Double/Capture/Furthest - same as capture, but prefer to land on a bonus spot, granting an extra turn (if possible)
- Capture/Double/Furthest - flip priority between capturing and landing on bonus spot
- Conservative - prefer to move away from spots where enemy pieces could capture on their next move land in spots where enemy pieces can't land

Next, I wanted to see how these strategies interacted with each other, and if there was a clear-cut best strategy that outperformed the others. But first, I wanted to make sure my strategies were actually working properly.

Game Visualization



I developed this tool (`run_pygame_match.py`) that would provide a visual simulation of a game of Trouble after selecting strategies by color.

Tournaments

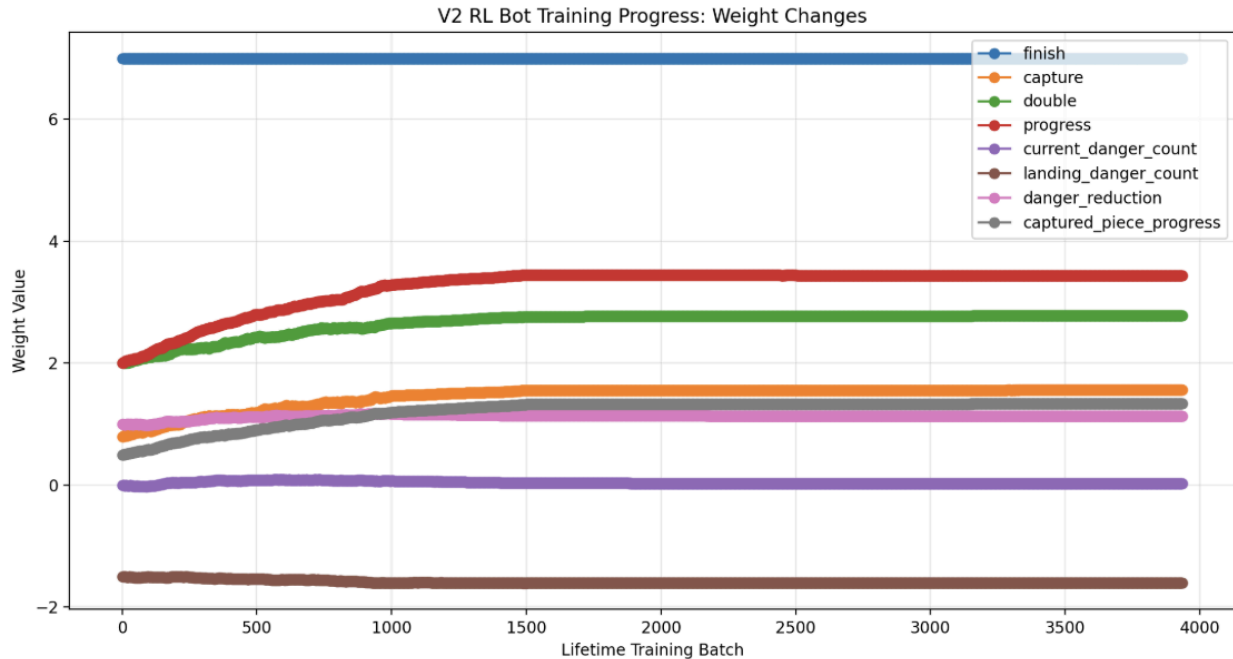
To determine which strategy performed the best, I decided to employ tournaments (`trouble_sim/tournaments.py`). Every strategy would play against every other one, and there'd be an equal distribution of games played in each starting position, as well as in 2/3/4 player games. The double/capture/furthest strategy and capture/double/furthest strategy dominated the other ones, and sized up pretty comparably when playing one another. The conservative strategy consistently placed third behind the two, and the random strategy placed last, unsurprisingly.

	A	B	C	D	E	F	G	H
1	strategy_a	strategy_b	num_games	avg_turns	max_turns_obse	strategy_a_win	strategy_b_win_rate	
2	random	random	1200	85.95916667	157	1	1	
3	random	furthest	2400	76.68375	126	0.235	0.765	
4	random	double_capture	2400	80.58583333	146	0.05	0.95	
5	random	capture_double	2400	81.11791667	169	0.04041666667	0.9595833333	
6	random	conservative	2400	83.34458333	154	0.08333333333	0.9166666667	
7								
8	furthest	furthest	1200	69.31	102	1	1	
9	furthest	double_capture	2400	73.08125	128	0.2391666667	0.7608333333	
10	furthest	capture_double	2400	73.72083333	125	0.22	0.78	
11	furthest	conservative	2400	72.80958333	119	0.2691666667	0.7308333333	
12								
13	double_capture	double_capture	1200	83.07416667	160	1	1	
14	double_capture	capture_double	2400	85.01625	169	0.4758333333	0.5241666667	
15	double_capture	conservative	2400	84.18375	177	0.60125	0.39875	
16								
17	capture_double	capture_double	1200	86.375	161	1	1	
18	capture_double	conservative	2400	85.40583333	166	0.63125	0.36875	
19								
20	conservative	conservative	1200	81.36166667	129	1	1	

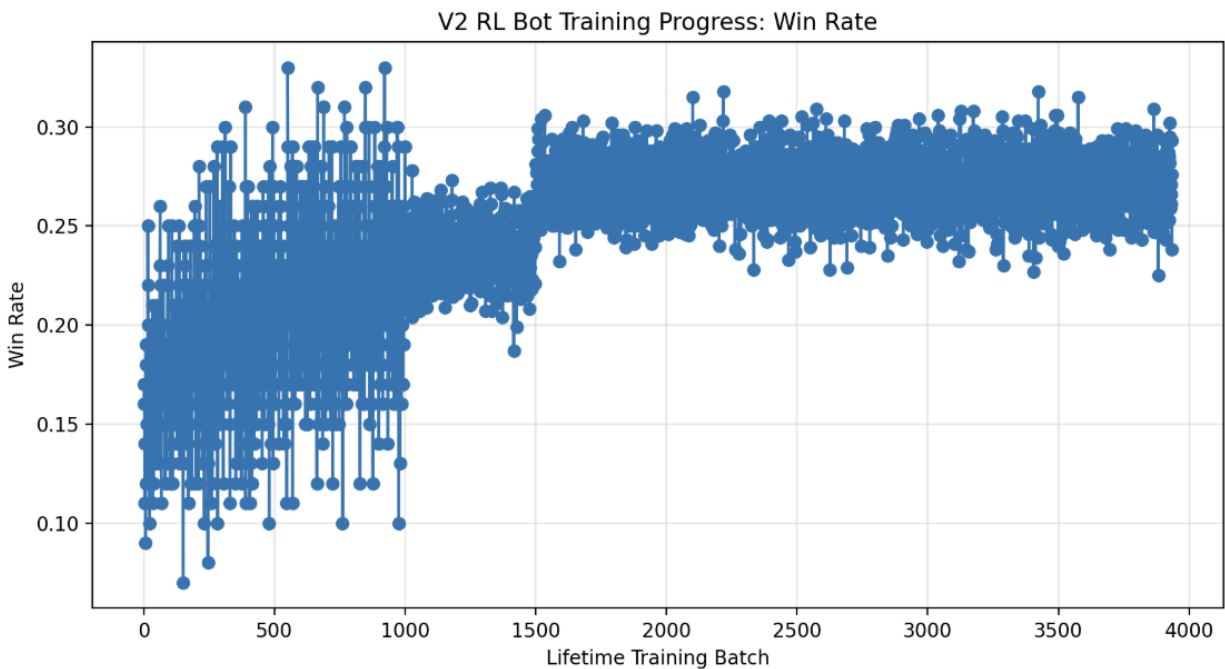
A sample from the results - 2 player games. Note the 96% win rate of the capture/double/furthest strategy vs the random strategy

Deep Learning Bot

The best strategies we've seen so far were heuristic-based, rewarding completing certain objectives. I wanted a more advanced strategy that would make these decisions based on the context of the game. I created a bot with preset preferences for certain scenarios, like moving a piece to the finish, capturing, or escaping danger. Rewards were given after each match based on number of pieces finished, captures, and overall progress of pieces. I realized it was difficult to train a bot in this context, as Trouble is as reliant on chance as you can get for board games – you could have a game where the bot theoretically makes all the best moves, yet still loses. I wanted to avoid those situations of rewarding games won by luck and punishing games lost despite good choices.



After running 4000 games against three bots employing the double/capture/furthest strategy, here were the changes in weight, which plateaued after roughly 1500 matches.



The reward structure was slightly tweaked around 1500 to favor outright winning the game more, which led to the jump in win rate around that time. The bot was able to win 27.6% of the last 500 training games it played. When testing it out of training mode, we also achieved a 27.6% win rate.

Conclusion

I would love to come back to this with a better approach to training the bot, given the random nature of the environment, but I really enjoyed the process. Going from a simple simulation of the game online, to learning how to train a bot and having it end up be slightly better than the best heuristic-driven strategies I could come up with was incredibly fun.